

Proyecto

Infraestructura para la
Computación de Altas Prestaciones
Curso: 2024/2025

Ángel González de Santos
Rubén Del Valle Carrillo

Infraestructura para la Computación de Altas Prestaciones

ÍNDICE

<i>Tarea 1</i>	3
<i>Tarea 2</i>	7
<i>Tarea 3</i>	16
<i>Anexo de automatización</i>	23
<i>Resumen de recursos y servicios desplegados</i>	34

Tarea 1

Primero, se debe crear una **tabla Dynamo DB** con clave de partición fecha y nombre 'proy-tabla'. Esta tabla tendrá cuatro columnas distintas, 'fecha' (que usaremos con indexación para que se diferencien los distintos valores de la tabla), 'mes_anyo' (para poder hacer una diferenciación mensual necesaria para posteriores apartados de la práctica), 'desviacion' y 'media'.

Hemos elegido 'fecha' clave de partición ya que necesitamos un valor que nos ayude a buscar días sueltos que nos ayudan a los cálculos necesarios para la siguiente tarea. Al principio pensamos en utilizar la variable 'mes_anyo' como clave de partición, pero debido a que nos dificulta a la hora de encontrar valores concretos preferimos dejarlo así.

Ahora creamos el **bucket S3** donde añadiremos el archivo de datos csv original. Lo llamaremos 'proy-ingestadatos'.

A continuación creamos la **notificación SNS**, la llamaremos 'proy-alarma-temperatura' y será del tipo Estándar. Le deberemos añadir una suscripción de protocolo email añadida a nuestro correo para comprobar que las notificaciones llegan correctamente.

Finalmente debemos crear las **dos funciones lambda** que manejarán nuestros datos y nos notificarán de ello. La primera de ella se llamará 'proy-funcion-procesador-datos' con Python 3.9, además deberemos cambiar el rol de ejecución a LabRole. Deberemos agregar como desencadenador el bucket s3 creado anteriormente para que pueda coger los datos de ahí cuando carguemos el archivo csv. También en configuración cambiaremos el tiempo de espera a 1 minuto para que le diese tiempo a seleccionar todos los datos ya que la función `datosTable.put_Item()` tarda demasiado tiempo con la cantidad de elementos que tenemos por lo que había que aumentar ese tiempo.

Esta función deberá seleccionar los valores brutos que encontramos en el archivo csv y limpiarlos para añadirlos a la tabla DynamoDB y además crear la variable 'mes_anyo' a partir de los valores de fecha.

Arquitectura:

```
# 1. Obtener detalles del archivo subido desde el evento S3
bucket_name = event['Records'][0]['s3']['bucket']['name']
file_key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

print(f"Procesando archivo: {file_key} del bucket: {bucket_name}")
```

- Obtenemos el nombre del bucket y la key para poder recoger la información del bucket S3 al que mandamos el archivo csv

Infraestructura para la Computación de Altas Prestaciones

2. Leer el archivo CSV desde S3

```
response = s3.get_object(Bucket=bucket_name, Key=file_key)
content = response['Body'].read().decode('utf-8')
```

3. Parsear el CSV

```
csv_reader = csv.DictReader(io.StringIO(content), delimiter=',') # Ajusta delimitador si es necesario
```

- Entramos al bucket S3 y leemos el archivo csv

```
for row in csv_reader:
    # Formato esperado del CSV: Fecha, Medias, Desviaciones
    fecha = row['Fecha'] # Ej: 22/03/2017
    media = row['Medias']
    desviacion = row['Desviaciones']

    partes_fecha = fecha.split('/')
    if len(partes_fecha) == 3:
        dia_normalizado = int(partes_fecha[2])
        mes_normalizado = int(partes_fecha[1])
        anio = partes_fecha[0]
        mes_ano = f"{mes_normalizado}-{anio}" # Ej: "03-2017" se convierte en "3-2017"
        fecha = f"{anio}/{mes_normalizado}/{dia_normalizado}" # Ej: "2017-03-22" se convierte en "2017-3-22"
    else:
        print(f"Formato de fecha incorrecto: {fecha}")
        continue
```

- Recorremos las filas del archivo y dentro de este seleccionamos las columnas 'Fecha', 'Medias' y 'Desviaciones'. Seguidamente dividimos el valor fecha en 'mes_normalizado' (normalizado para que sea 3 en vez de 03), 'dia_normalizado'(igual que mes) y 'anio' y así crea la variable mes_ano que necesitaremos más adelante y poder normalizar la variable fecha.

4. Insertar en DynamoDB

```
try:
    datosTable.put_item(
        Item={
            'mes_ano': mes_ano,
            'fecha': fecha, #Partition key
            'media': media,
            'desviacion': desviacion
        }
    )
    items_processed += 1
except Exception as e:
    print(f"Error insertando {fecha}: {str(e)}")
```

- Intenta añadir los valores sacados del bucket en la tabla DynamoDB creada, sino es posible devolverá un error.

Infraestructura para la Computación de Altas Prestaciones

Para la segunda función le pondremos de nombre 'proy-funcion-notificacion' con las mismas características iniciales que la función anterior. A esta función le agregaremos como desencadenador la tabla DynamoDB creada para poder seleccionar los valores de desviación seleccionados, además le pondremos como tamaño del lote 500 para asegurarnos que coge todos los datos.

Esta función deberá mandar una notificación al usuario suscrito al tema creado cuando se encuentre una desviación superior a 0.5, de este modo le llegará una notificación con la fecha y la desviación.

Arquitectura:

```
# For each inventory item added, check if the count is zero
for record in event['Records']:
    if record['eventName'] == 'INSERT':
        try:
            newImage = record['dynamodb']['NewImage']

            desviacion_str = newImage['desviacion']['S']
            fecha = newImage['fecha']['S']
            desviacion_val = float(desviacion_str)
```

- Revisa cada registro de eventos recibidos. Solo actuará si se añade un evento (record['eventName'] == 'INSERT'). Intenta obtener la nueva imagen del dato insertado y saca la desviación (que convierte luego a número float) y la fecha, sino lo puede hacer devuelve un error

```
if desviacion_val > 0.5:
    mensaje = (
        f"ALERTA AQUASENSE:\n"
        f"Fecha: {fecha}\n"
        f"Desviación detectada: {desviacion_val}\n"
        f"Umbral permitido: 0.5"
    )
    sns = boto3.client('sns')

    alertTopic = 'proy-alarma-temperatura'

    snsTopicArn = [t['TopicArn'] for t in sns.list_topics()['Topics']
                    if t['TopicArn'].lower().endswith(':' + alertTopic.lower())]
    for i in range(0, len(snsTopicArn)):

        sns.publish(
            TopicArn=snsTopicArn[i],
            Message=mensaje,
            Subject='¡Alerta Crítica Mar Menor!'
        )
        print(f"Alerta enviada para fecha {fecha}")
        alertas_enviadas += 1
```

Infraestructura para la Computación de Altas Prestaciones

- Si la desviación es mayor que 0.5 (condición de ejecutar esta función) crea un mensaje con la fecha y la desviación detectada, y selecciona el tema creado en SNS y lo publica enviando este mensaje a todos los usuarios suscritos al tema creado.

Como hemos considerado diferenciar entre los aspectos del proyecto que conllevan a las practicas 2 y 4, ahora haremos una comprobación de que al llegar a este punto todo ha funcionado bien. Comenzamos añadiendo distintas particiones del archivo csv original para comprobar que la tabla no se sobrecarga.

☐	fecha (Cadena)	desviacion	media	mes_año
☐	2018/04/17	0.24110382...	17.312650...	4-2018
☐	2023/2/14	0.28404419...	11.479106...	2-2023
☐	2021/3/10	0.41271377...	15.007667...	3-2021
☐	2021/4/27	0.15223167...	18.302458...	4-2021
☐	2017/06/02	0.27056753...	24.581281...	6-2017
☐	2021/1/27	0.56915503...	12.332708...	1-2021
☐	2019/11/21	0.21803041...	13.575810...	11-2019
☐	2021/3/15	0.52441862...	16.591060...	3-2021
☐	2022/2/16	0.13143265...	13.417865...	2-2022
☐	2021/12/15	0.11531102...	12.972942...	12-2021
☐	2021/8/24	0.27845760...	28.599952...	8-2021
☐	2023/12/7	0.30889326...	14.622605...	12-2023
☐	2017/05/04	0.27256551...	19.501352...	5-2017
☐	2023/3/1	0.33037279...	11.916785...	3-2023
☐	2022/12/1	0.40661169...	15.569854...	12-2022

Como podemos ver en la captura se han cargado todos los datos (son 380 ya que hay algunas filas que están repetidas y se sobrescriben).

¡Alerta Crítica Mar Menor! Recibidos x



AWS Notifications <no-reply@sns.amazonaws.com>
para mí ▾

ALERTA AQUASENSE:

Fecha: 2017/04/05

Desviación detectada: 0.6253794431686401

Umbral permitido: 0.5

—

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:

<https://sns.us-east-1.amazonaws.com/unsubscribe.html?SubscriptionArn=arn:aws:sns:us-east-1:949076425070:proy-alarma-temperatura:f1e30053-a030-4f57-ba98-a9cca5c0b852&Endpoint=angelg6605@gmail.com>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

15:05 (hace 3 horas)

También en esta captura podemos ver que hemos recibido la notificación oportuna para un caso concreto en el que la desviación supera el umbral. En total llegan 36 correos por lo que suponemos que puede haber 36 casos en los que se supera este umbral.

Infraestructura para la Computación de Altas Prestaciones

Tarea 2

Una vez realizado todo lo necesario para construir la arquitectura del pipeline, va a ser necesario crear un servicio web al que hacer las solicitudes, este servicio cogerá los datos provenientes de la tabla de DynamoDB creada anteriormente.

1º Creación de infraestructura básica a través de plantilla:

Accediendo a Cloudformation, creamos una nueva pila llamada proy-pila la cual creará lo establecido en la plantilla “proy-plantilla”.

- Empezamos creando una VPC de nombre proy-vpc la cual contendrá dos subredes públicas llamadas proy-public-subnet-1 y proy-public-subnet-2, estas subredes se encontrarán en zonas de disponibilidad distintas para garantizar alta disponibilidad y tolerancia a fallos.

```
4 Resources:
5
6 ## Crear la VPC
7 VPC:
8   Type: AWS::EC2::VPC
9   Properties:
10     CidrBlock: 192.168.0.0/16
11     EnableDnsSupport: true
12     EnableDnsHostnames: true
13     Tags:
14       - Key: Name
15         Value: proy-vpc
16
17 ## Subred Pública 1
18 PublicSubnet1:
19   Type: AWS::EC2::Subnet
20   Properties:
21     VpcId: !Ref VPC
22     CidrBlock: 192.168.1.0/24
23     AvailabilityZone: !Select
24       - 0
25       - !GetAZs
26         Ref: AWS::Region
27     MapPublicIpOnLaunch: true
28     Tags:
29       - Key: Name
30         Value: proy-public-subnet-1
31
32 ## Subred Pública 2
33 PublicSubnet2:
34   Type: AWS::EC2::Subnet
35   Properties:
36     VpcId: !Ref VPC
37     CidrBlock: 192.168.2.0/24
38     AvailabilityZone: !Select
39       - 1
40       - !GetAZs
41         Ref: AWS::Region
42     MapPublicIpOnLaunch: true
43     Tags:
44       - Key: Name
45         Value: proy-public-subnet-2
```

- Creamos una tabla de enrutamiento, la cual contendrá las reglas necesarias para el correcto tráfico de información, creando a su vez una ruta de acceso a internet Gateway. Asociamos las dos subredes a la tabla de enrutamiento, haciendo que sean accesibles por internet.

```
47 ## Tabla de rutas pública
48 PublicRouteTable:
49   Type: AWS::EC2::RouteTable
50   Properties:
51     VpcId: !Ref VPC
52
53 ## Ruta de internet
54 PublicRoute:
55   Type: AWS::EC2::Route
56   DependsOn: VPCGatewayAttachment
57   Properties:
58     RouteTableId: !Ref PublicRouteTable
59     DestinationCidrBlock: 0.0.0.0/0
60     GatewayId: !Ref InternetGateway
61
62 ## Asociar tabla de rutas con subred 1
63 PublicRouteTableAssociation:
64   Type: AWS::EC2::SubnetRouteTableAssociation
65   Properties:
66     SubnetId: !Ref PublicSubnet1
67     RouteTableId: !Ref PublicRouteTable
68
69 ## Asociar tabla de rutas con subred 2
70 PublicRouteTableAssociation2:
71   Type: AWS::EC2::SubnetRouteTableAssociation
72   Properties:
73     SubnetId: !Ref PublicSubnet2
74     RouteTableId: !Ref PublicRouteTable
75
76 ## Internet Gateway
77 InternetGateway:
78   Type: AWS::EC2::InternetGateway
79   Properties: {}
80
81 ## Adjuntar IGW a la VPC
82 VPCGatewayAttachment:
83   Type: AWS::EC2::VPCGatewayAttachment
84   Properties:
85     VpcId: !Ref VPC
86     InternetGatewayId: !Ref InternetGateway
```

Infraestructura para la Computación de Altas Prestaciones

- Creamos tres grupos de seguridad cada uno con una función diferente:

FlaskSecurityGroup: Permite tráfico entrante al puerto 5000 y al puerto 22, ira asociado a la instancia docker, permitiendo que esta sea accesible a través de internet (puerto 5000) y accesible a través de Putty gracias al puerto 22. Nos será necesario para el desarrollo de la aplicación Flask.

```
107 ##Grupo de seguridad asociado a la instancia
108 FlaskSecurityGroup:
109   Type: AWS::EC2::SecurityGroup
110   Properties:
111     GroupDescription: Permite conexión a la instancia
112     VpcId: !Ref VPC
113     SecurityGroupIngress:
114       - IpProtocol: tcp
115         FromPort: 5000
116         ToPort: 5000
117         CidrIp: 0.0.0.0/0
118       - IpProtocol: tcp
119         FromPort: 22
120         ToPort: 22
121         CidrIp: 0.0.0.0/0
122     Tags:
123       - Key: Name
124         Value: Flask Security Group
125
```

BalanceadorSecurityGroup: El principal objetivo es permitir que el tráfico web estándar llegue al balanceador

TaskSecurityGroup: Este grupo de seguridad únicamente permite el tráfico en el puerto 5000 proveniente del balanceador de carga.

```
107 ## Security Group del Balanceador
108 BalanceadorSecurityGroup:
109   Type: AWS::EC2::SecurityGroup
110   Properties:
111     GroupName: BalanceadorSecurityGroup
112     GroupDescription: "Acceso HTTP publico para balanceador"
113     VpcId: !Ref VPC
114     SecurityGroupIngress:
115       - IpProtocol: tcp
116         FromPort: 80
117         ToPort: 80
118         CidrIp: 0.0.0.0/0
119     Tags:
120       - Key: Name
121         Value: BalanceadorSecurityGroup
122
123 ## Security Group de las tareas ECS (puerto 5000 desde el balanceador)
124 TaskSecurityGroup:
125   Type: AWS::EC2::SecurityGroup
126   DependsOn: BalanceadorSecurityGroup
127   Properties:
128     GroupName: TaskSecurityGroup
129     GroupDescription: "Permite trafico al puerto Flask 5000 desde el balanceador"
130     VpcId: !Ref VPC
131     SecurityGroupIngress:
132       - IpProtocol: tcp
133         FromPort: 5000
134         ToPort: 5000
135         SourceSecurityGroupId: !Ref BalanceadorSecurityGroup
136     Tags:
137       - Key: Name
138         Value: TaskSecurityGroup
139
```

- Finalmente, creamos una instancia de EC2 llamada DockerInstance con un par de claves asociado llamado proy-keypair, que habrá sido creado previamente. Como he mencionado antes, tendrá asociado el grupo de seguridad FlaskSecurityGroup.

La finalidad de hacer de esta instancia un docker, es hacerla un entorno de prueba y host de contenedores, aprovechando la portabilidad, el aislamiento y la eficiencia que ofrecen estos. DiskVolume básicamente servirá para expandir la capacidad del docker y DiskMountPoint para que el docker acceda a ese espacio.

Infraestructura para la Computación de Altas Prestaciones

```
140 ## Instancia EC2 de pruebas
141 DockerInstance:
142   Type: AWS::EC2::Instance
143   Properties:
144     InstanceType: t2.micro
145     ImageId: ami-08a0d1e16fc3f61ea
146     KeyName: proy-keypair
147     IamInstanceProfile: LabInstanceProfile
148     SubnetId: !Ref PublicSubnet1
149     SecurityGroupIds:
150       - !Ref FlaskSecurityGroup
151     Tags:
152       - Key: Name
153         Value: DockerInstance
154     UserData:
155       Fn::Base64: !Sub |
156         #!/bin/bash
157         dnf update -y
158         dnf install -y docker
159         systemctl enable docker
160         systemctl start docker
161   DiskVolume:
162     Type: AWS::EC2::Volume
163     Properties:
164       Size: 100
165       AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
166     DeletionPolicy: Snapshot
167   DiskMountPoint:
168     Type: AWS::EC2::VolumeAttachment
169     Properties:
170       InstanceId: !Ref DockerInstance
171       VolumeId: !Ref DiskVolume
172       Device: /dev/sdh
173
174
```

2º Creación de aplicación web:

El siguiente paso es comenzar a crear la aplicación Flask:

- Accedemos a putty con la Ip publica de DockerInstance y el par de claves proy-keypair.
- Creamos un nuevo directorio el cual contendrá la aplicación web flask llamado flask_docker. Este directorio tendrá tres ficheros: Dockerfile, requirements.txt y función.py

Funcion.py: El objetivo de la función Python contenido en este fichero será recibir solicitudes web y responder a éstas.

En este programa Python, primero importamos la clase Flask desde el paquete flask y lo utilizamos para crear una instancia de aplicación Flask llamada app.

Creo una interfaz orientada a objetos para interactuar con el servicio de DynamoDB. Este objeto (dynamodb) es el punto de partida para acceder a todas las tablas que existan en esa región. Una vez establecida la conexión accedo a la tabla que cree al principio 'proy-tabla'.

```
GNU nano 5.8
from flask import Flask, request, jsonify
import boto3
from decimal import Decimal
from boto3.dynamodb.conditions import Attr

app = Flask(__name__)

tabla_dy = 'proy-tabla'
region = 'us-east-1'

dynamodb = boto3.resource('dynamodb', region_name=region)
table = dynamodb.Table(tabla_dy)
```

Infraestructura para la Computación de Altas Prestaciones

Creo una función auxiliar llamada “media” la cual nos será útil en la función “get_temp()”, toma como parámetros un diccionario llamado ítems y el campo al que queremos acceder del objeto en cuestión.

```
def media(items, field: str):  
    """  
    Calcula la media de una lista numérica almacenada en el atributo `field`  
    (lista de Decimal / float / int).  
    """  
    # Convertimos cada valor a float por si vienen como Decimal  
    valores = []  
    for i in items:  
        valores.append(float(i[field]))  
  
    media_global = sum(valores) / len(valores)  
    return media_global
```

El decorador @app.route transforma la función Python en una función “view” de Flask que convierte la respuesta de la función en una respuesta HTTP. Definiremos las rutas de la función /temp, /sd, /maxdiff las cuales permitirán acceso a las funciones en cuestión. Solo responderá a solicitudes GET, que son típicamente utilizadas para recuperar datos.

Esta línea recupera dos parámetros de la URL: year (año) y month (mes).

```
y, m = request.args.get('year'), request.args.get('month')
```

Creamos una clave de búsqueda en el formato mes-año, que nos permitirá acceder a la columna “mes_ano” de la tabla de dynamodb más fácilmente. Con “table.scan” se filtra para buscar solo aquellos ítems donde el atributo “mes-ano” sea igual al valor buscado. Con “response.get('Items',[])” guardamos en un diccionario todos los ítems que coincidan. Para majear los fallos debidos a la solicitud de datos inexistentes en nuestra tabla agregamos una excepción.

```
mes_ano = f"{m}-{y}"  
response = table.scan(  
    FilterExpression=Attr("mes_ano").eq(mes_ano)  
)  
items = response.get('Items', [])  
if not items:  
    return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)
```

- get_temp()

Esta función le aplica la función media al campo “media” a todos los datos de un mes, devolviéndonos la media de temperatura de un mes.

Infraestructura para la Computación de Altas Prestaciones

```
@app.route('/temp', methods=['GET'])
def get_temp():
    y, m = request.args.get('year'), request.args.get('month')
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )
    items = response.get('Items', [])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    media_temp = media(items, "media")

    return jsonify({
        f"Media de temperatura del mes {mes_ano}": media_temp
    })
```

- get_maxdiff():

En esta función va a calcular la máxima diferencia de temperatura que existe entre el mes solicitado y el mes inmediatamente anterior. La función primero calcula el máximo del primer mes y luego el máximo del segundo, luego calcula la diferencia.

```
GNU nano 5.8                                funcion.py
@app.route('/maxdiff', methods=['GET'])
def get_maxdiff():
    y, m = int(request.args.get('year')), int(request.args.get('month'))
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )
    items = response.get('Items', [])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    max_act = 0
    for i in items:
        if float(i['media']) > max_act:
            max_act = float(i['media'])

    if m == 1:
        m_2 = 12
        y_2 = y-1
    else:
        m_2 = m-1
        y_2 = y

    mes_ano_ant = f"{m_2}-{y_2}"
    response_ant = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano_ant)
    )
    items_ant = response_ant.get('Items', [])
    if not items_ant:
        return jsonify({"error": f"Datos no encontrados para mes anterior"}, 404)

    max_ant = 0
    for i in items_ant:
        if float(i['media']) > max_ant:
            max_ant = float(i['media'])

    max_diff = max_act-max_ant
    return jsonify({ f"Diferencia maxima de temperatura del mes {mes_ano}": max_diff
```

- get_sd():

Esta función devuelve el valor máximo del campo “desviación” de un mes:

Infraestructura para la Computación de Altas Prestaciones

```
@app.route('/sd', methods=['GET'])
def get_sd():
    y, m = request.args.get('year'), request.args.get('month')
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )

    items = response.get('Items', [])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    max = 0
    for i in items:
        if float(i['desviacion']) > max:
            max = float(i['desviacion'])

    return jsonify({
        f"Maxima desviacion del mes {mes_ano}": max
    })
```

- health_check():

Este es un endpoint de salud, utilizado para comprobar si la aplicación esta en funcionamiento.

```
@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200
```

Las dos últimas líneas de este programa garantizan por un lado que si el puerto 5000 está ya ocupado por otra aplicación el programa siga ejecutándose en otro puerto. Por otro lado, la aplicación Flask se ejecuta en modo depuración y es accesible desde el exterior

```
if __name__ == '__main__':
```

```
    app.run(host='0.0.0.0', port=5000)
```

*Es posible que al copiar la función este código de error, por lo que podría ser necesario reescribirlo a mano.

requirements.txt: Fichero que contiene todas las dependencias de la aplicación Flask:

Infraestructura para la Computación de Altas Prestaciones

```
GNU nano 5.8
Flask==2.1.1
boto3==1.24.5
werkzeug==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.1
itsdangerous==2.1.2
click==8.1.3
```

Dockerfile:

- Utiliza una imagen base basada en Linux Alpine (debido a su tamaño reducido) que será el sistema operativo en el que instalaremos nuestra imagen.
- Establece el directorio /app como el directorio de trabajo de nuestra aplicación en el contenedor.
- Copia el fichero requirements.txt en el directorio de trabajo del contenedor.
- Instala todos los paquetes indicados en el fichero requirements.txt.
- Copia todo el contenido del directorio actual y el directorio de trabajo.
- Indica el comando que ejecutará el contenedor.

```
GNU nano 5.8 Dockerfile
# Usamos la imagen base de Python 3.8 sobre Alpine
FROM python:3.8-alpine

# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos el archivo requirements.txt al contenedor
COPY requirements.txt requirements.txt

# Instalamos las dependencias listadas en el requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos todo el contenido de la carpeta local al contenedor
COPY . .

# Especificamos el comando para ejecutar el script Python metadata.py al iniciar
CMD ["python3", "funcion.py"]
```

3º Prueba de funcionamiento:

- Construimos la imagen y creamos el contenedor en el puerto 5000

Infraestructura para la Computación de Altas Prestaciones

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker build -t flask_container .
[+] Building 0.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 641B
=> [internal] load metadata for docker.io/library/python:3.8-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.8-alpine@sha256:3d93b1f77efce339aa77db726656872517b0d67837989aa7c4b35bd5ae7e81ba
=> [internal] load build context
=> => transferring context: 368B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt requirements.txt
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:64d6e714ac76fd323c2a3d20e8ab73e7485f2a456d0b68596dff5a8f640630db
=> => naming to docker.io/library/flask_container
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker run -p 5000:5000 -d flask_container
d2192de50a78b358efe7655af54771dc7b77af100c1df799960620eb061aa614
```

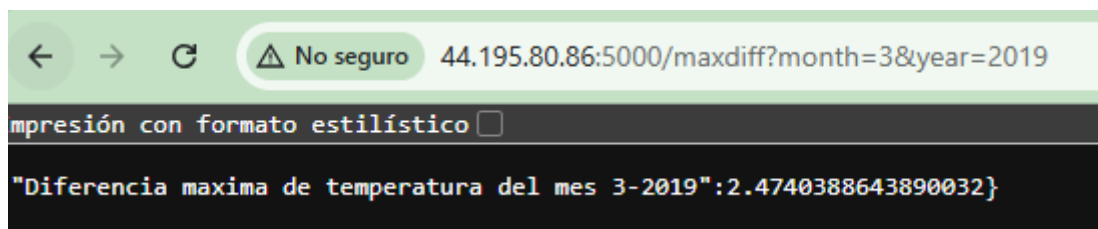
- Comprobamos que se han creado correctamente:

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
d2192de50a78   flask_container  "python3 funcion.py"    14 seconds ago Up 13 seconds  0.0.0.0:5000->5000/tcp, :::5000->5000/tcp  heuristic_maxwell
```

```
[ec2-user@ip-192-168-1-33 flask_docker]$ sudo docker images
REPOSITORY      TAG        IMAGE ID      CREATED        SIZE
flask_container  latest     64d6e714ac76  23 minutes ago  132MB
```

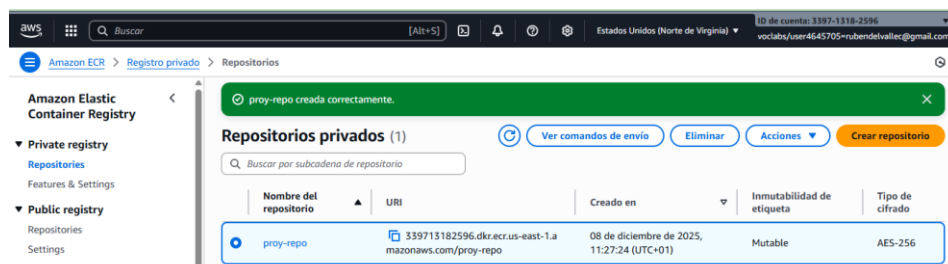
- Comprobamos que todas las funciones funcionan correctamente tanto en la terminal como desde internet:

```
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/sd?month=10&year=2019"
{"Maxima desviacion del mes 10-2019":0.6360582330690658}
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/temp?month=10&year=2019"
{"Media de temperatura del mes 10-2019":22.658597559121585}
[ec2-user@ip-192-168-1-33 flask_docker]$ curl "http://127.0.0.1:5000/maxdiff?month=10&year=2019"
{"Diferencia maxima de temperatura en mes 10-2019":5.310085582610821}
[ec2-user@ip-192-168-1-33 flask_docker]$
```



4º Creación del repositorio al que subir la imagen

- Como paso previo a la tarea3, creamos un repositorio ECR al cual he llamado “proy-repo”, cuya funcionalidad será almacenar, gestionar y distribuir imágenes de contenedores.



Infraestructura para la Computación de Altas Prestaciones

Subimos la imagen al repositorio:

- Recupero un token de autenticación que autentique nuestro cliente de Docker en el registro.

```
[ec2-user@ip-192-168-1-33 ~]$ aws ecr get-login-password --region us-east-1 | do  
cker login --username AWS --password-stdin 339713182596.dkr.ecr.us-east-1.amazon  
aws.com
```

- Etiqueto la imagen para poder enviarla a este repositorio:

```
[ec2-user@ip-192-168-1-33 ~]$ docker tag flask_container 339713182596.dkr.ecr.us  
-east-1.amazonaws.com/proy-repo:flask_container
```

- Envío esta imagen al repositorio de AWS recién creado

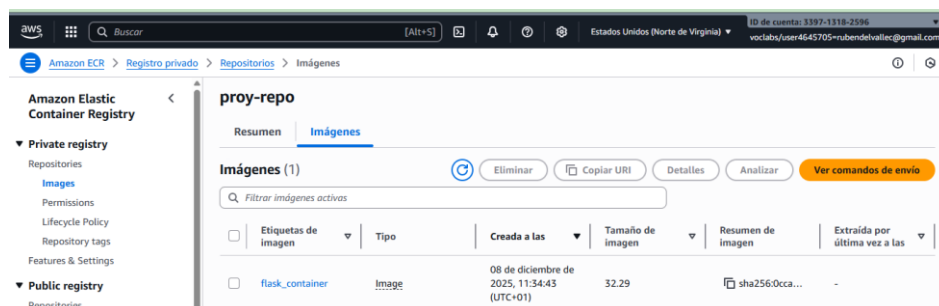
```
[ec2-user@ip-192-168-1-33 ~]$ docker push 339713182596.dkr.ecr.us-east-1.amazona  
ws.com/proy-repo:flask_container
```

```
[ec2-user@ip-192-168-1-33 ~]$ aws ecr get-login-password --region us-east-1 | do  
cker login --username AWS --password-stdin 339713182596.dkr.ecr.us-east-1.amazon  
aws.com  
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/conf  
ig.json.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/engine/reference/commandline/login/#credentials-store  
  
Login Succeeded  
[ec2-user@ip-192-168-1-33 ~]$ docker tag flask_container 339713182596.dkr.ecr.us  
-east-1.amazonaws.com/proy-repo:flask_container  
[ec2-user@ip-192-168-1-33 ~]$ docker push 339713182596.dkr.ecr.us-east-1.amazona  
ws.com/proy-repo:flask_container  
The push refers to repository [339713182596.dkr.ecr.us-east-1.amazonaws.com/proy  
-repo]  
746cc873e8d6: Pushed  
9ad62875e95c: Pushed  
6c677fe46a63: Pushed  
268865912762: Pushed  
817fa7f5a4af: Pushed  
6b49371d7ed7: Pushed  
50acbb6dc641: Pushed  
63calfbb43ae: Pushed  
flask_container: digest: sha256:0cca7ca33b52a8f336a0877a62ca51fbbbd912dd272a4156  
0c4f48a2f6b904f2 size: 1990
```

*Es posible que al intentar acceder a nuestro token de autenticación debido a que el sistema entiende que no tenemos permisos, por lo que puede ser útil copiar y volver a ingresar a esta ahora si con permisos:

```
sudo usermod -aG docker ec2-user
```

Finalmente, comprobamos que se ha subido correctamente.



Infraestructura para la Computación de Altas Prestaciones

Tarea 3

Ahora deberemos ocuparnos de crear e implementar las aplicaciones en un entorno de contenedores. Para esto usaremos la imagen de Docker creada.

Primero deberemos crear **una puerta de enlace(Punto de conexión)** entre la vpc y dynamo DB para facilitar el paso de información de la tabla 'proy-tabla'. Debemos entrar en puntos de conexión en VPC y crear uno. Le llamaremos 'proy-vpc-dynamodb'. La categoría del servicio será Servicio AWS. En Servicios deberemos buscar dynamo y seleccionaremos la opción 'com.amazonaws.us-east-1.dynamodb' con el tipo Gateway. Para finalizar seleccionaremos la vpc 'proy-vpc' y todas las tablas de enrutamiento que aparecen (dos).

[Alt+S]

Estados Unidos (Norte de Virginia)

ID de cuenta: 8517-2562-7517

voclabs/user4645605-AngelGonzalez2

Punto de enlace de la VPC creado correctamente
vpce-036f3adbbefa214da

Puntos de enlace (1/1) Información

Buscar puntos de conexión por atributo o etiqueta

ID de punto de enlace de la VPC: vpce-036f3adbbefa214da

Borrar filtros

Name	ID de punto de enlace de la ...	Tipo de punto de enlace	Estado	Nombre del servicio	Red de servicio
proy-vpc-dynamodb	vpce-036f3adbbefa214da	Gateway	Disponible	com.amazonaws.us-east-1.dynamodb	-

vpce-036f3adbbefa214da / proy-vpc-dynamodb

Detalles Tablas de enrutamiento Política Etiquetas

Detalles

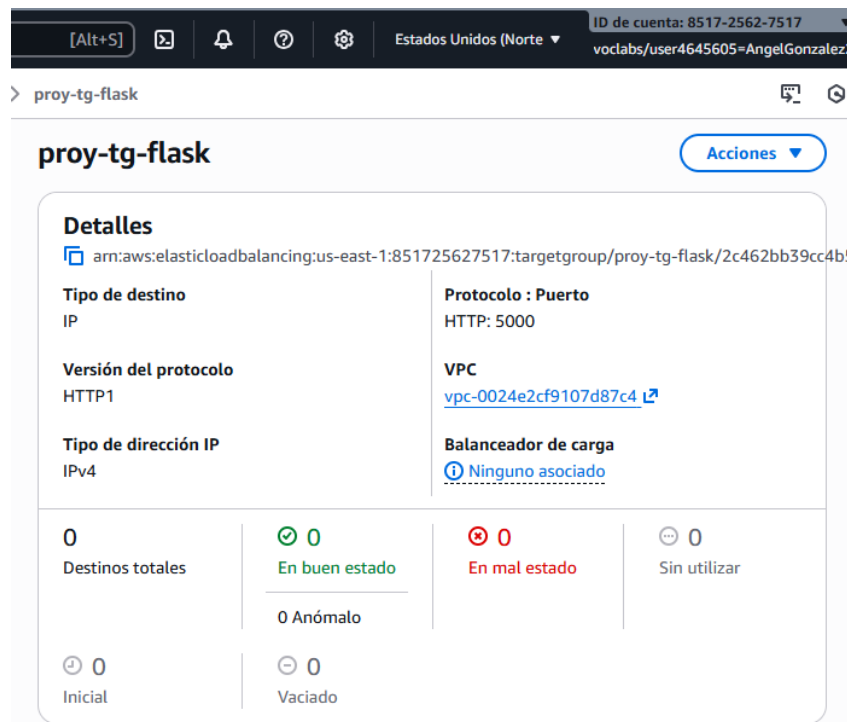
ID de punto de enlace vpce-036f3adbbefa214da	Estado Disponible	Hora de creación lunes, 8 de diciembre de 2025, 13:13:42 CET	Tipo de punto de enlace Gateway
ID de la VPC vpc-0024e2cf9107d87c4 (proy-vpc)	Mensaje de estado -	Nombre del servicio com.amazonaws.us-east-1.dynamodb	Nombres de DNS privados habilitados No
Tipo de IP de registro DNS service-defined	Tipo de dirección IP ipv4	Región de servicio us-east-1	Preferencia de DNS privado -
Dominios DNS privados especificados -			

Comenzamos creando el **balanceador de carga** al cual llamaremos 'proy-alb'. Será un balanceador de carga de aplicaciones ya que vamos a usar un tráfico HTTP y HTTPS. Estará expuesto a Internet. Necesitamos que el balanceador escale dentro de una vpc, así que elegiremos la creada por la plantilla. Las zonas de disponibilidad deberán ser las dos subredes conectadas a la vpc. Claramente añadiremos como grupo de seguridad el que está relacionado con el balanceador (BalanceadorSecurityGroup). Crearemos el protocolo http y puerto 80 para el agente de escucha.

Finalmente faltaría añadir un **grupo de destino**, pero como no tenemos ninguno creado deberemos hacerlo. El grupo de destino se llamará 'proy-tg-flask'. El tipo de destino serán direcciones IP, esto lo hacemos ya usamos el modo de red aws vpc y cada contenedor tendrá su propia IP. El protocolo será HTTP y con puerto 5000 para coincidir con el puerto donde se escucha la aplicación Flask. La vpc (aunque está automáticamente seleccionada) será la creada en la plantilla. En la pestaña de comprobación de estado deberemos añadir 'health' a la ruta, ya que como hemos visto en la función de la tarea anterior nuestra función health_check que hace una comprobación de estado funciona con esta ruta. Finalmente deberemos modificar el umbral en buen estado como 2 (con 2

Infraestructura para la Computación de Altas Prestaciones

respuestas buenas seguidas el balanceador empieza a funcionar) y el intervalo a 6 (de este modo se realizan comprobaciones rápidas para detectar fallos o recuperaciones rápido)



proy-tg-flask Acciones

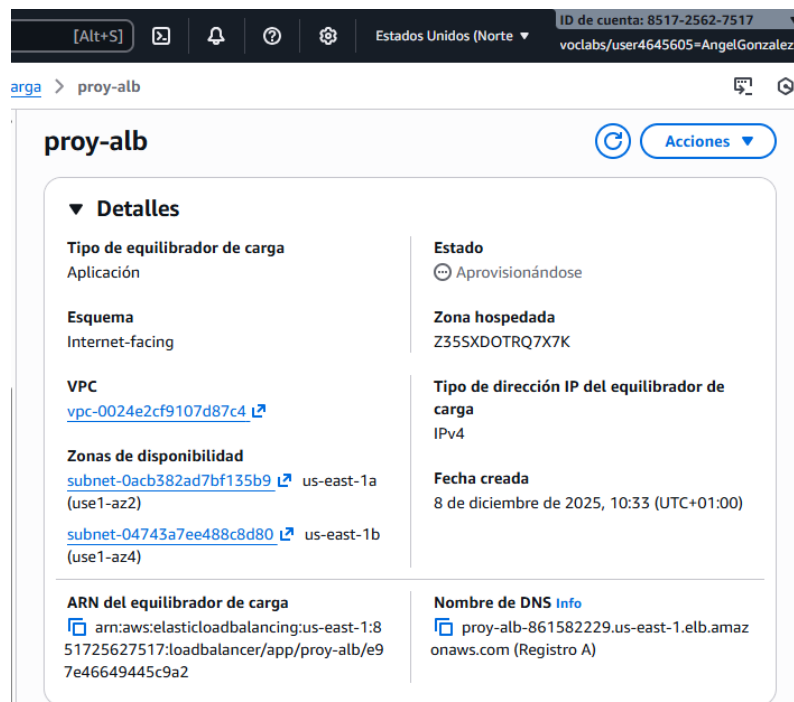
Detalles

arn:aws:elasticloadbalancing:us-east-1:851725627517:targetgroup/proy-tg-flask/2c462bb39cc4b5

Tipo de destino IP	Protocolo : Puerto HTTP: 5000
Versión del protocolo HTTP1	VPC vpc-0024e2cf9107d87c4
Tipo de dirección IP IPv4	Balanceador de carga Ninguno asociado

0 Destinos totales	0 En buen estado	0 En mal estado	0 Sin utilizar
0 Inicial	0 Vaciado	0 Anómalo	

Finalmente seleccionamos el grupo de destino creado para el balanceador y terminamos la implementación de este.



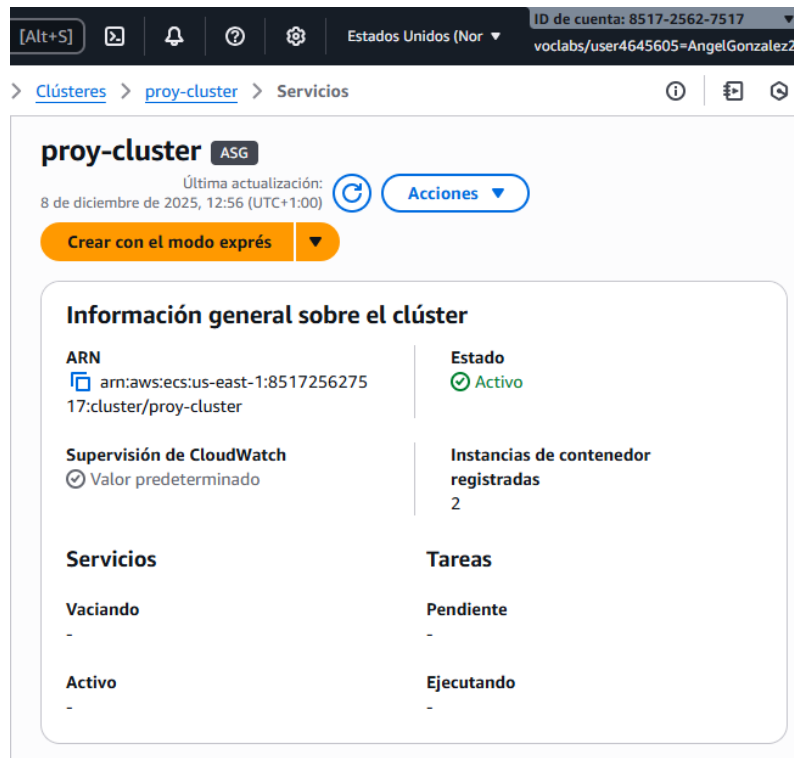
proy-alb Acciones

Detalles

Tipo de equilibrador de carga Aplicación	Estado Aprovisionándose
Esquema Internet-facing	Zona hospedada Z35SXDOTRQ7X7K
VPC vpc-0024e2cf9107d87c4	Tipo de dirección IP del equilibrador de carga IPv4
Zonas de disponibilidad subnet-0acb382ad7bf135b9 us-east-1a (use1-az2) subnet-04743a7ee488c8d80 us-east-1b (use1-az4)	Fecha creada 8 de diciembre de 2025, 10:33 (UTC+01:00)
ARN del equilibrador de carga arn:aws:elasticloadbalancing:us-east-1:851725627517:loadbalancer/app/proy-alb/e97e46649445c9a2	Nombre de DNS Info proy-alb-861582229.us-east-1.elb.amazonaws.com (Registro A)

Infraestructura para la Computación de Altas Prestaciones

A continuación crearemos el **clúster** con nombre 'proy-cluster'. La infraestructura será *Fargate e instancias autoadministrativas*. El modelo de aprovisionamiento deberá ser Bajo Demanda. El sistema operativo se queda como Amazon Linux 2023, pero el tipo de instancia será t2.micro ya que este tamaño es suficiente y económico para los contenedores de este proyecto. El rol de la instancia deberá ser LabInstanceProfile. En las capacidades deseadas la mínima debe ser 2 (garantiza que si una instancia falla, el servicio continúa con la otra) y la máxima 4 (Permite crecer automáticamente ante picos de demanda). Seleccionamos el par de claves creado para la pila y las vpc y subredes creadas en ella. Finalmente seleccionamos el grupo de seguridad TaskSecurityGroup y en la opción de asignar automáticamente una IP pública seleccionamos Encender, si no habilitamos esta opción, la instancia no tendrá una dirección IP accesible desde internet, por lo que no podremos conectarnos a ella mediante SSH ni ver la página web desde el navegador.



The screenshot shows the AWS Management Console interface for the 'proy-cluster' ASG. The top navigation bar includes the AWS logo, account ID (8517-2562-7517), and user information (voclabs/user4645605=AngelGonzalez2). The breadcrumb trail indicates the path: Clústeres > proy-cluster > Servicios. The main content area shows the 'proy-cluster' ASG with a status of 'Activo' (Active). Below this, there is a section titled 'Información general sobre el clúster' (General information about the cluster) which includes the ARN (arn:aws:ecs:us-east-1:8517256275:17:cluster/proy-cluster), the state (Activo), and the number of registered container instances (2). A table below this section lists the services and tasks for the cluster.

Información general sobre el clúster	
ARN	Estado
arn:aws:ecs:us-east-1:8517256275:17:cluster/proy-cluster	Activo
Supervisión de CloudWatch	Instancias de contenedor registradas
Valor predeterminado	2
Servicios	Tareas
Vaciando	Pendiente
-	-
Activo	Ejecutando
-	-

Seguidamente creamos la **definición de tareas** con nombre 'proy-task', el tipo de lanzamiento será únicamente Amazon EC2 ya que usamos un clúster con instancias EC2. Debemos confirmar que el modo de red es awsvpc ya que otorga a la tarea las mismas propiedades de red que las instancias EC2. Los tamaños serán 1 vCPU y .2 GB ya que la instancia t2.micro tiene recursos limitados y debemos dejar un margen para el sistema operativo. El rol de tarea y de ejecución corresponderán al LabRole.

Deberemos detallar el **contenedor**, al que llamaremos 'proy-container'. Copiamos en el apartado 'Imagen' la URI de la imagen que tenemos en el repositorio. En el mapeo de puertos el puerto del contenedor deberá ser 5000 y el protocolo TCP. Finalmente, los límites de asignación serán para la CPU 1, el límite estricto de memoria .2 GB y el límite flexible de memoria .1 GB (el GPU no lo tocamos).

Infraestructura para la Computación de Altas Prestaciones

The screenshot shows the AWS Management Console interface for a task definition named 'proy-task:1'. The top navigation bar includes the AWS logo, account ID (8517-2562-7517), and user information (voclabs/user4645605=AngelGonzalez2). The breadcrumb trail indicates the path: ... > proy-task > Revisión 1 > Contenedores. The main content area displays the task definition details, including the ARN, creation time, role, operating system, and network mode. The status is 'ACTIVE'.

Información general	
ARN arn:aws:ecs:us-east-1:8517256275:17:task-definition/proy-task:1	Estado ACTIVE
Hora de creación 8 de diciembre de 2025, 12:55 (UTC+1:00)	Entorno de la aplicación EC2
Rol de tarea LabRole	Rol de ejecución de tareas LabRole
Sistema operativo/arquitectura Linux/X86_64	Modo de red awsvpc
Inyección de errores Desactivado	

Finalmente creamos el **servicio**, este lo haremos dentro del clúster creado anteriormente. Le llamaremos 'proy-ecs-service'. Debemos seleccionar la definición de tareas 'proy-task' creada anteriormente. Como tipo de lanzamiento seleccionaremos EC2. El tipo de servicio será RÉPLICA ya que así mantendremos el número de tareas que hemos seleccionado en el clúster si hay algún fallo. Las tareas deseadas son 2 hecha para la tolerancia a fallos. Seleccionamos la vpc creada y las subredes relacionadas. El grupo de seguridad deberá ser TaskSecurityGroup ya que este grupo solo permite tráfico en el puerto 5000 proveniente del Balanceador de Carga. A continuación debemos seleccionar en la pestaña 'Equilibrio de carga' que vamos a usar uno. Será un balanceador de carga de aplicaciones y en contenedor seleccionamos el creado 'proy-container 5000:5000', el balanceador de carga será 'proy-alb', el agente de escucha el ya creado 'HTTP:80' y el grupo de destino será 'proy-tg-flask'.

Ahora empezamos a hacer comprobaciones de que todo ha ido bien:

- Las pilas relacionadas con el servicio y el clúster se han creado sin problemas

Infraestructura para la Computación de Altas Prestaciones

[Alt+S] [Iconos] Estados Unidos (Norte de Virgi) ID de cuenta: 8517-2562-7517
voclabs/user4645605=AngelGonzalez2

Pilas (4) [Icono] [Eliminar] [Actualizar pila] [Acciones de pila] [Crear pila]

Q Buscar por nombre de pila

Filtrar estado
Activo [Vista anidada]

	Nombre de la pila	Estado	Hora de creación
☐	ECS-Console-V2-Service-proy-task-service-proy-cluster-2d6a9d4c	CREATE_COMPLETE	2025-12-08 13:36
☐	Infra-ECS-Cluster-proy-cluster-20495a66	CREATE_COMPLETE	2025-12-08 12:05
☐	proy-pila	CREATE_COMPLETE	2025-12-07 23:28
☐	c177448a4586040113025588t1w851725627517	CREATE_COMPLETE	2025-12-07 16:45

- Las tareas se han ejecutado correctamente y el servicio se mantiene activo

[Alt+S] [Iconos] Estados Unidos (Norte de Virginia) ID de cuenta: 8517-2562-7517
voclabs/user4645605=AngelGonzalez2

> Clústeres > proy-cluster > Servicios > proy-task-service > Tareas

proy-task-service Información
Última actualización: 8 de diciembre de 2025, 13:40 (UTC+1:00) [Icono] [Eliminar el servicio] [Actualizar servicio]

Descripción general del servicio Información

Estado Activo	Tareas (2 deseadas) 0 pendiente/s 2 en ejecución	Definición de tarea: revisión proy-task:1	Estado de despliegue Realizado correctamente
-------------------------	--	--	--

< Estado y métricas **Tareas** Registros Implementaciones Eventos Configuración y red >

Tareas (1/2) Última actualización: 8 de diciembre de 2025, 13:40 (UTC+1:00) [Icono] [Detener]

Q Filtrar tareas por propiedad [Filtrar estado deseado: Cualquier estado deseado] [Filtrar tipo de lanzamiento: Cualquier tipo de lanzamiento]

	Tarea	Último est...	Estado de...	D...	Estado	Creado a las
☐	b7b2367148a34376be73...	Ejecutando	Ejecutando	pro...	Desconocido	hace 3 minutos
☒	fe52e59bf79c43298f619...	Ejecutando	Ejecutando	pro...	Desconocido	hace 4 minutos

- El grupo de destino tiene dos destinos en buen estado

Infraestructura para la Computación de Altas Prestaciones

[Alt+S] [Notificaciones] [Ayuda] [Configuración] Estados Unidos (Norte de Virginia) ID de cuenta: 8517-2562-7517 voclabs/user4645605=AngelGonzalez2

> proy-tg-flask  

proy-tg-flask Acciones ▼

Detalles

 [arn:aws:elasticloadbalancing:us-east-1:851725627517:targetgroup/proy-tg-flask/aa3e089772e7bfc4](#)

Tipo de destino IP	Protocolo : Puerto HTTP: 5000	Versión del protocolo HTTP1	VPC vpc-0024e2cf9107d87c4 
Tipo de dirección IP IPv4	Balanceador de carga proy-alb 		

2 Destinos totales	 2 En buen estado 0 Anómalo	 0 En mal estado	 0 Sin utilizar	 0 Inicial	 0 Vaciado
-----------------------	--	--	---	---	--

► **Distribución de destinos por zona de disponibilidad (AZ)**
Seleccione los valores de esta tabla para ver los filtros correspondientes aplicados a la tabla Destinos registrados que aparece a continuación.

Destinos | Monitorización | Comprobaciones de estado | Atributos | Etiquetas

Destinos registrados (2) [Info](#)  **Mitigación de anomalías: No aplicable**  Anular el registro Registrar destinos

Los grupos de destinos enrutan las solicitudes a destinos individuales registrados mediante el protocolo y el número de puerto que especifique. Las comprobaciones de estado se realizan en todos los destinos registrados de acuerdo con la configuración de comprobación de estado del grupo de destinos. La detección de anomalías se aplica automáticamente a los grupos de destinos de HTTP/HTTPS con al menos 3 destinos en buen estado.

< 1 > 

☐	dirección IP	Pue...	Zona	Estado	Detalles del estado	Sustitu...	Detalle...
☐	192.168.1.109	5000	us-east-1a (...)	 Healthy	-	 No override.	No overri...
☐	192.168.2.26	5000	us-east-1b (...)	 Healthy	-	 No override.	No overri...

- Se han creado dos instancias, si además entramos a las dos instancias las direcciones IP de los destinos registrados son las direcciones IPv4 privadas

Infraestructura para la Computación de Altas Prestaciones

Instancias (1/3) Información

Conectar Estado de la instancia Acciones

Lanzar instancias

Buscar Instancia por atributo o etiqueta (case-sensitive) Todos los estados

Name	ID de la instancia	Estado de la i...	Tipo de inst...
ECS Instance - ...	i-0bedfc84fc7c08535	En ejecución	t2.micro
DockerInstance	i-0b190c597ed9b214b	En ejecución	t2.micro
ECS Instance - ...	i-00a2ebb12576916ea	En ejecución	t2.micro

i-00a2ebb12576916ea (ECS Instance - proy-cluster)

Detalles Estado y alarmas Monitoreo Seguridad Redes

Resumen de instancia Información

ID de la instancia
i-00a2ebb12576916ea

Direcciones IPv4 privadas
192.168.1.185
192.168.1.109

Estado de la instancia
En ejecución

Dirección IPv4 pública
18.206.64.235 | dirección abierta

Dirección IPv6
-

DNS público
ec2-18-206-64-235.compute-1.amazonaws.com | dirección abierta

- Finalmente iremos solicitando en el navegador que todo va bien:

```
← → ↻ No es seguro proy-alb-1935258244.us-east-1.elb.amazonaws.com/temp?month=1&year=2020
Dar formato al texto
{"Media de temperatura del mes 1-2020":12.324003940717382}
```

```
← → ↻ No es seguro proy-alb-1935258244.us-east-1.elb.amazonaws.com/sd?month=4&year=2018
Dar formato al texto
{"Maxima desviacion del mes 4-2018":0.4494665861129761}
```

```
← → ↻ No es seguro proy-alb-751991610.us-east-1.elb.amazonaws.com/maxdiff?month=7&year=2017
Impresión con formato estilístico
{"Diferencia maxima de temperatura del mes 7-2017":1.2870254516601562}
```

Infraestructura para la Computación de Altas Prestaciones

- También lo hacemos para los errores

```
← → ↻ ⚠ No es seguro proy-alb-1935258244.us-east-1.elb.amazonaws.com/sd?month=8&year=2025
Dar formato al texto ☐
[{"error": "Datos no encontrados para 8-2025"}, 404]
```

```
← → ↻ ⚠ No es seguro proy-alb-1935258244.us-east-1.elb.amazonaws.com/temp?month=1&year=2026
Dar formato al texto ☐
[{"error": "Datos no encontrados para 1-2026"}, 404]
```

```
← → ↻ ⚠ No es seguro proy-alb-1935258244.us-east-1.elb.amazonaws.com/maxdiff?month=1&year=2027
Dar formato al texto ☐
[{"error": "Datos no encontrados para 1-2027"}, 404]
```

Anexo de automatización

- Función lambda 'Proy-funcion-procesador-datos':

```
import json, urllib.parse
import boto3
import csv
import io
import os

# Clientes de AWS
s3 = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')
datosTable = dynamodb.Table('proy-tabla')

def lambda_handler(event, context):

    # 1. Obtener detalles del archivo subido desde el evento S3
    bucket_name = event['Records'][0]['s3']['bucket']['name']
    file_key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])

    print(f"Procesando archivo: {file_key} del bucket: {bucket_name}")
```

Infraestructura para la Computación de Altas Prestaciones

```
# 2. Leer el archivo CSV desde S3
response = s3.get_object(Bucket=bucket_name, Key=file_key)
content = response['Body'].read().decode('utf-8')

# 3. Parsear el CSV
csv_reader = csv.DictReader(io.StringIO(content), delimiter=',') # Ajusta
delimitador si es necesario

items_processed = 0

for row in csv_reader:
    # Formato esperado del CSV: Fecha, Medias, Desviaciones
    fecha = row['Fecha'] # Ej: 22/03/2017
    media = row['Medias']
    desviacion = row['Desviaciones']

    partes_fecha = fecha.split('/')
    if len(partes_fecha) == 3:
        dia_normalizado = int(partes_fecha[2])
        mes_normalizado = int(partes_fecha[1])
        anio = partes_fecha[0]
        mes_ano = f"{mes_normalizado}-{anio}" # Ej: "03-2017" se convierte en
"3-2017"
        fecha = f"{anio}/{mes_normalizado}/{dia_normalizado}" # Ej: "2017/03/22"
se convierte en "2017/3/22"
    else:
        print(f"Formato de fecha incorrecto: {fecha}")
        continue

# 4. Insertar en DynamoDB
try:
    datosTable.put_item(
        Item={
            'fecha': fecha,          # Partition Key
            'mes_ano': mes_ano,
            'media': media,
            'desviacion': desviacion
        }
    )
    items_processed += 1
except Exception as e:
    print(f"Error insertando {fecha}: {str(e)}")

return {
```

Infraestructura para la Computación de Altas Prestaciones

```
'statusCode': 200,
'body': json.dumps(f'Procesados {items_processed} registros.')
```

- Función lambda 'Proy-funcion-notificacion':

```
# Stock Check Lambda function
#
# This function is triggered when values are inserted into the Inventory DynamoDB
# table.
# Inventory counts are checked and if an item is out of stock, a notification is
# sent to an SNS Topic.
import json, boto3

# This handler is run every time the Lambda function is triggered
def lambda_handler(event, context):
    # Show the incoming event in the debug log
    print("Event received by Lambda function: " + json.dumps(event, indent=2))
    alertas_enviadas = 0
    # For each inventory item added, check if the count is zero
    for record in event['Records']:
        if record['eventName'] == 'INSERT':
            try:
                newImage = record['dynamodb']['NewImage']

                desviacion_str = newImage['desviacion']['S']
                fecha = newImage['fecha']['S']
                desviacion_val = float(desviacion_str)

                if desviacion_val > 0.5:
                    mensaje = (
                        f"ALERTA AQUASENSE:\n"
                        f"Fecha: {fecha}\n"
                        f"Desviación detectada: {desviacion_val}\n"
                        f"Umbral permitido: 0.5"
                    )
                    sns = boto3.client('sns')
                    alertTopic = 'proy-alarma-temperatura'

                    snsTopicArn = [t['TopicArn'] for t in
sns.list_topics()['Topics']]
```

Infraestructura para la Computación de Altas Prestaciones

```

        if t['TopicArn'].lower().endswith(': ' +
alertTopic.lower())]
        for i in range(0, len(snsTopicArn)):

            sns.publish(
                TopicArn=snsTopicArn[i],
                Message=mensaje,
                Subject='¡Alerta Crítica Mar Menor!'
            )
            print(f"Alerta enviada para fecha {fecha}")
            alertas_enviadas += 1

    except KeyError as e:
        print(f"Falta algún campo en el registro: {e}")
    except ValueError as e:
        print(f"Error convirtiendo desviación a número: {e}")

# Finished!
return 'Successfully processed {} records.'.format(len(event['Records']))

```

- Pila CloudFormation :

```

AWSTemplateFormatVersion: '2010-09-09'
Description: Crear una VPC con dos subredes públicas, grupos de seguridad y una
instancia EC2

Resources:

  ## Crear la VPC
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: 192.168.0.0/16
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: proy-vpc

  ## Subred Pública 1
  PublicSubnet1:
    Type: AWS::EC2::Subnet
    Properties:

```

Infraestructura para la Computación de Altas Prestaciones

```
VpcId: !Ref VPC
CidrBlock: 192.168.1.0/24
AvailabilityZone: !Select
- 0
- !GetAZs
  Ref: AWS::Region
MapPublicIpOnLaunch: true
Tags:
- Key: Name
  Value: proy-public-subnet-1

## Subred Pública 2
PublicSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: 192.168.2.0/24
    AvailabilityZone: !Select
    - 1
    - !GetAZs
      Ref: AWS::Region
    MapPublicIpOnLaunch: true
    Tags:
    - Key: Name
      Value: proy-public-subnet-2

## Tabla de rutas pública
PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

## Ruta de internet
PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: VPCGatewayAttachment
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref InternetGateway

## Asociar tabla de rutas con subred 1
PublicRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
```

Infraestructura para la Computación de Altas Prestaciones

```
SubnetId: !Ref PublicSubnet1
RouteTableId: !Ref PublicRouteTable

## Asociar tabla de rutas con subred 2
PublicRouteTableAssociation2:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet2
    RouteTableId: !Ref PublicRouteTable

## Internet Gateway
InternetGateway:
  Type: AWS::EC2::InternetGateway
  Properties: {}

## Adjuntar IGW a la VPC
VPCGatewayAttachment:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref VPC
    InternetGatewayId: !Ref InternetGateway
##Grupo de seguridad asociado a la instancia
FlaskSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Enable TCP ingress for Flask
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 5000
        ToPort: 5000
        CidrIp: 0.0.0.0/0
      - IpProtocol: tcp
        FromPort: 22
        ToPort: 22
        CidrIp: 0.0.0.0/0
    Tags:
      - Key: Name
        Value: Flask Security Group

## Security Group del Balanceador
BalanceadorSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
```

Infraestructura para la Computación de Altas Prestaciones

```
GroupName: BalanceadorSecurityGroup
GroupDescription: "Acceso HTTP publico para balanceador"
VpcId: !Ref VPC
SecurityGroupIngress:
  - IpProtocol: tcp
    FromPort: 80
    ToPort: 80
    CidrIp: 0.0.0.0/0
Tags:
  - Key: Name
    Value: BalanceadorSecurityGroup

## Security Group de las tareas ECS (puerto 5000 desde el balanceador)
TaskSecurityGroup:
  Type: AWS::EC2::SecurityGroup
  DependsOn: BalanceadorSecurityGroup
  Properties:
    GroupName: TaskSecurityGroup
    GroupDescription: "Permite trafico al puerto Flask 5000 desde el balanceador"
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 5000
        ToPort: 5000
        SourceSecurityGroupId: !Ref BalanceadorSecurityGroup
    Tags:
      - Key: Name
        Value: TaskSecurityGroup

## Instancia EC2 de pruebas
DockerInstance:
  Type: AWS::EC2::Instance
  Properties:
    InstanceType: t2.micro
    ImageId: ami-08a0d1e16fc3f61ea
    KeyName: proy-keypair
    IamInstanceProfile: LabInstanceProfile
    SubnetId: !Ref PublicSubnet1
    SecurityGroupIds:
      - !Ref FlaskSecurityGroup
    Tags:
      - Key: Name
        Value: DockerInstance
  UserData:
    Fn::Base64: !Sub |
```

Infraestructura para la Computación de Altas Prestaciones

```
#!/bin/bash
dnf update -y
dnf install -y docker
systemctl enable docker
systemctl start docker

DiskVolume:
  Type: AWS::EC2::Volume
  Properties:
    Size: 100
    AvailabilityZone: !GetAtt DockerInstance.AvailabilityZone
    DeletionPolicy: Snapshot

DiskMountPoint:
  Type: AWS::EC2::VolumeAttachment
  Properties:
    InstanceId: !Ref DockerInstance
    VolumeId: !Ref DiskVolume
    Device: /dev/sdh

Outputs:

VPCId:
  Description: ID de la VPC creada
  Value: !Ref VPC

PublicSubnet1Id:
  Description: "Subred publica 1"
  Value: !Ref PublicSubnet1

PublicSubnet2Id:
  Description: "Subred pública 2"
  Value: !Ref PublicSubnet2

TestInstancePublicIP:
  Description: "IP publica de la instancia EC2 de pruebas"
  Value: !GetAtt DockerInstance.PublicIp
```

- **Shell scripts de AWS CLI :**

```
#Crear archivos
mkdir flask_docker
nano flask_docker/funcion.py
nano flask_docker/requirements.txt
nano flask_docker/Dockerfile
```

Infraestructura para la Computación de Altas Prestaciones

```
#Probar imagen
cd flask_docker
sudo systemctl start docker #inicio docker
sudo docker build -t flask_container . #Construyo una imagen Docker
sudo docker run -p 5000:5000 -d flask_container #Inicio un contenedor Docker a
partir de la imagen flask_container
sudo docker ps
curl "http://127.0.0.1:5000/maxdiff?month=3&year=2017" #Compruebo que toda va bien

##subir imagen a ECR

#Recupere un token de autenticación y autentique su cliente de Docker en el
registro. Use AWS CLI:
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin 339713182596.dkr.ecr.us-east-1.amazonaws.com

#Cuando se complete la creación, etiquete la imagen para poder enviarla a este
repositorio:
docker tag flask_container 339713182596.dkr.ecr.us-east-1.amazonaws.com/proy-
repo:flask_container

#comando para enviar esta imagen al repositorio de AWS recién creado
docker push 339713182596.dkr.ecr.us-east-1.amazonaws.com/proy-repo:flask_container
```

- Función de AWS CLI 'funcion.py':

```
from flask import Flask, request, jsonify
import boto3
from decimal import Decimal
from boto3.dynamodb.conditions import Attr

app = Flask(__name__)

tabla_dy = 'proy-tabla'
region = 'us-east-1'

dynamodb = boto3.resource('dynamodb', region_name=region)
table = dynamodb.Table(tabla_dy)

def media(items, field: str):
    """
    Calcula la media de una lista numérica almacenada en el atributo `field`
    (lista de Decimal / float / int).
    """
```

Infraestructura para la Computación de Altas Prestaciones

```
# Convertimos cada valor a float por si vienen como Decimal
valores = []
for i in items:
    valores.append(float(i[field]))

media_global = sum(valores) / len(valores)
return media_global

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy'}), 200

@app.route('/temp', methods=['GET'])
def get_temp():
    y, m = request.args.get('year'), request.args.get('month')
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )

    items = response.get('Items', [])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    media_temp = media(items, "media")

    return jsonify({
        f"Media de temperatura del mes {mes_ano}": media_temp
    })

@app.route('/maxdiff', methods=['GET'])
def get_maxdiff():
    y, m = int(request.args.get('year')), int(request.args.get('month'))
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )
    items = response.get('Items', [])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    max_act = 0
    for i in items:
        if float(i['media']) > max_act:
```

Infraestructura para la Computación de Altas Prestaciones

```
        max_act = float(i['media'])

    if m == 1:
        m_2 = 12
        y_2 = y-1

    else:
        m_2 = m-1
        y_2 = y

    mes_ano_ant = f"{m_2}-{y_2}"
    response_ant = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano_ant)
    )
    items_ant = response_ant.get('Items',[])
    if not items_ant:
        return jsonify({"error": f"Datos no encontrados para mes anterior"}, 404)

    max_ant = 0
    for i in items_ant:
        if float(i['media']) > max_ant:
            max_ant = float(i['media'])

    max_diff = max_act-max_ant
    return jsonify({
        f"Diferencia maxima de temperatura del mes {mes_ano}": max_diff
    })

@app.route('/sd', methods=['GET'])
def get_sd():
    y, m = request.args.get('year'), request.args.get('month')
    mes_ano = f"{m}-{y}"
    response = table.scan(
        FilterExpression=Attr("mes_ano").eq(mes_ano)
    )
    items = response.get('Items',[])
    if not items:
        return jsonify({"error": f"Datos no encontrados para {mes_ano}"}, 404)

    max = 0
    for i in items:
        if float(i['desviacion']) > max:
            max = float(i['desviacion'])

    return jsonify({
```

Infraestructura para la Computación de Altas Prestaciones

```
f"Maxima desviacion del mes {mes_anyo}": max
})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

- Documentos de AWS CLI 'requirements.txt' y 'Dockerfile':

```
Flask==2.1.1
boto3==1.24.5
werkzeug==2.1.2
Jinja2==3.1.2
MarkupSafe==2.1.1
itsdangerous==2.1.2
click==8.1.3
```

```
# Usamos la imagen base de Python 3.8 sobre Alpine
FROM python:3.8-alpine

# Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos el archivo requirements.txt al contenedor
COPY requirements.txt requirements.txt

# Instalamos las dependencias listadas en el requirements.txt
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos todo el contenido de la carpeta local al contenedor
COPY . .

# Especificamos el comando para ejecutar el script Python metadata.py al inicia>
CMD ["python3", "funcion.py"]
```

Resumen de recursos y servicios desplegados

Identificador de recurso	Nombre de recurso	Tipo de recurso en AWS	Comentario (opcional)
--------------------------	-------------------	------------------------	-----------------------

Infraestructura para la Computación de Altas Prestaciones

vpc-0842e39f3eb0d667a	proy-vpc	VPC	Red principal
subnet-0d5cc650df98dd98c	Proy-public-subnet-1	Subred	Subred publica
subnet-06aee243cbaf21fdf	Proy-public-subnet-2	Subred	Subred publica
i-06cc1064baccd8e67	DockerInstance	Instancia de EC2	Docker
i-0c8f6658282a5c38a	ECS-instance-proy-cluster	Instancia de EC2	Instancia creada con el servicio
i-04739e1f15e19cb2a	ECS-instance-proy-cluster	Instancia de EC2	Instancia creada con el servicio
sg-0a9d6cb1394d451f5	BalanceadroSecurityGroup	Grupo de seguridad	Grupo de seguridad asociado al balanceador
sg-0e5102c2d3fa77f5b	TaskSecurityGroup	Grupo de seguridad	Grupo de seguridad asociado a las tareas
sg-09db73553c9d47671	FlaskSecurityGroup	Grupo de Seguridad	Grupo de seguridad asociado al Docker
proy-alb-1715306209.us-east-1.elb.amazonaws.com	Proy-alb	Balanceador de carga	Redistribuye el trabajo
arn:aws:elasticloadbalancing:us-east-1:851725627517:targetgroup/proy-tg-flask/e9ab6f881f6314c1	Proy-tg-flask	Grupo de destino	Redirecciona el trafico
vpce-00744cb23b647b8db	Proy-vpc-dynamodb	Punto de conexion	Conecta la vpc con dynamodb
851725627517.dkr.ecr.us-east-1.amazonaws.com/proy-repo	Proy-repo	Repositorio	Almacena imagenes

Infraestructura para la Computación de Altas Prestaciones

arn:aws:ecs:us-east-1:851725627517:cluster/proy-cluster	Proy-cluster	Cluster	Grupo de instancias que ejecuta contenedores
	Proy-keypair	Par de claves	
arn:aws:s3:::proy-ingestadatos	Proy-ingestadatos	Bucket de S3	Almacena datos en sucio
arn:aws:lambda:us-east-1:851725627517:function:proy-funcion-procesador-datos	Proy-funcion-procesador-datos	Funcion Lambda	Proceso los datos situados en el proy-ingestadatos y modifica para ponerlos en el formato correcto
arn:aws:lambda:us-east-1:851725627517:function:proy-funcion-notificacion	Proy-funcion-notificacion	Funcion Lambda	Detecta desviación mayor que 0.5
arn:aws:sns:us-east-1:851725627517:proy-alarma-temperatura	Proy-alarma-temperatura	Tema SNS	Envia mensajes a suscriptores
arn:aws:ecs:us-east-1:851725627517:task-definition/proy-task:2	Proy-task	Tarea	Encapsula lo necesario para ejecutar contenedor
arn:aws:cloudformation:us-east-1:851725627517:stack/proy-pila/d44dae80-d758-11f0-a270-12c75d78730d	Proy-pila	Pila de CloudFormation	Lanza infraestructura basica
arn:aws:cloudformation:us-east-1:851725627517:stack/ECS-Console-V2-Service-proy-ecs-service-proy-cluster-e4703a46/0c8e97c0-d75c-11f0-87d2-0affe5514849	ECS-Console-V2-Service-proy-ecs-service-proy-cluster-e4703a46	Pila de CloudFormation	Lanza servicio

Infraestructura para la Computación de Altas Prestaciones

rtb-032cc2a96131b2cd8	-	Tabla de enrutamiento	Asocia las dos subredes a la VPC
arn:aws:dynamodb:us-east-1:851725627517:table/proy-tabla	Proy-tabla	Tabla DynamoDB	Tabla de DynamoDB donde guardamos información
arn:aws:ecs:us-east-1:851725627517:service/proy-cluster/proy-ecs-service	proy-ecs-service	Servicio	Servicio que ejecuta tareas para el balanceador de carga